



Universidad Nacional Experimental de Guayana

Vicerrectorado Académico

Coordinación General de Pregrado

Proyecto de Carrera: Ingeniería en Informática

Unidad Curricular: Lenguaje y Compiladores

Los lenguajes de programación

Equipo: **VISIONLY**

Mantenlo simple!

Profesor:

Felix Marquez

Integrantes:

Ana Santamaria, V-21251440

Carlos Martínez V-27955202

Genesis Moya V-31370339

Naleska Carrera V-31521506

Ciudad Guayana, junio 3 del 2026

RESUMEN

El presente estudio analiza la infraestructura lingüística y computacional de los lenguajes de programación modernos desde la perspectiva del diseño de compiladores. El propósito fundamental es evaluar de forma analítica y profunda el impacto de las estructuras morfológicas, léxicas y sintácticas sobre la eficiencia, expresividad y mantenibilidad del software en entornos de procesamiento intensivo.

Para ello, se estructuró una matriz descriptiva que aborda el fenómeno contemporáneo de la convergencia multiparadigma en los modelos imperativo, orientado a objetos, funcional, lógico y concurrente. Asimismo, se ejecutó un proceso de benchmarking técnico bajo entornos controlados de hardware para medir comparativamente los tiempos de ejecución y el consumo de memoria pico de cuatro lenguajes con filosofías arquitectónicas profundamente diferenciadas: Zig, Rust, Python y JavaScript. Los resultados empíricos validan cómo la compilación nativa frente a los mecanismos interpretados o de ejecución Just-In-Time (JIT) condicionan críticamente el rendimiento del software ante algoritmos de alta carga de procesamiento.

Finalmente, para consolidar la visión de los lenguajes como productos arquitectónicos orientados a fines específicos, se diseñó conceptualmente el "Lenguaje L", un Lenguaje de Dominio Específico (DSL) adaptado para la gestión del sistema lógico-físico crítico "ECO-GRID" (Microredes Eléctricas Inteligentes y Almacenamiento de Energía).

Tras la definición formal de su alfabeto, reglas léxicas y gramática sintáctica abstracta, se demostró la viabilidad de la propuesta mediante el desarrollo de dos escenarios operativos: la prevención autónoma de fugas térmicas en bancos de baterías y la optimización de balance de carga para la inyección de excedentes a la red pública.

INTRODUCCIÓN

El presente informe tiene como objetivo principal examinar detenidamente la naturaleza estructural de los lenguajes de programación a través de sus aspectos morfológicos, léxicos y sintácticos. Para alcanzar este objetivo, el informe se ha organizado en una secuencia de tres fases tanto analíticas como prácticas.

Se explora el análisis de los paradigmas de programación, estableciendo cómo diferentes enfoques teóricos influyen en la manera en que los lenguajes se ocupan de la solución de problemas. A continuación, se lleva a cabo un estudio analítico y de referencia que compara lenguajes actuales y de gran relevancia en la industria moderna (como Zig, Rust, Python y JavaScript). Esta evaluación permite entender de manera directa cómo cambian las estructuras gramaticales, las intenciones de diseño y la administración de recursos según la arquitectura de cada lenguaje.

Toda la fundamentación teórica analizada se une en un ejercicio riguroso de ingeniería aplicada: el diseño conceptual del "Lenguaje L". Este Lenguaje de Dominio Específico (DSL) ha sido creado desde cero para la automatización, la prevención de errores y la gestión comercial del entorno energético simulado ECO-GRID. A través de la colaboración en grupo, se definen el alfabeto, los tokens y la gramática sintáctica abstracta que proporcionan a este lenguaje una coherencia lógica indiscutible para la toma de decisiones.

Actividad 1: Matriz Descriptiva de Paradigmas de Programación.

A continuación, se detalla la estructura conceptual de cada paradigma, analizada a través de sus ejes temáticos fundamentales.

- Paradigma Imperativo / Estructural

Este paradigma es el más cercano a la arquitectura de von Neumann, basado en el concepto del cambio de estado a través de sentencias.

- Gestión explícita del estado del sistema: El programador tiene control directo sobre el estado de la máquina en un momento dado, definiendo el "cómo" se deben hacer las cosas paso a paso.
- Secuenciación de instrucciones: La ejecución sigue un flujo de control lineal y estructurado (iteraciones, selecciones, secuencias).
- Mutabilidad de memoria: Las variables son representaciones directas de celdas de memoria cuyos valores cambian a lo largo de la ejecución.
- Efectos secundarios (Side effects): Las funciones o rutinas pueden modificar el estado global del programa, lo que, aunque es eficiente a nivel de hardware, dificulta el rastreo de errores en sistemas complejos (Sebesta, 2019).

- Paradigma Orientado a Objetos (POO)

Surge para organizar el código imperativo en sistemas grandes, agrupando datos y comportamiento.

- Modelos de encapsulamiento: Ocultamiento de la representación interna del estado del objeto; el acceso se realiza estrictamente a través de interfaces públicas (métodos).
- Polimorfismo: Capacidad de una entidad (interfaz o clase base) de presentar múltiples formas y comportamientos en tiempo de ejecución, facilitando la escalabilidad del código.

- Herencia vs. Composición: Transición moderna del diseño de jerarquías de clases rígidas (herencia "es-un") hacia patrones basados en ensamblaje de módulos (composición "tiene-un") para evitar alta acoplamiento.
 - Abstracción (datos y comportamiento): Modelado del mundo real donde las entidades son sujetos activos (objetos) que mantienen su propio estado y reglas de negocio asociadas (Ghezzi et al., 2002).
- Paradigma Funcional

Basado en el cálculo lambda, prioriza la evaluación de expresiones matemáticas puras sobre la ejecución de comandos.

- Inmutabilidad de datos: Una vez creadas, las estructuras de datos no pueden ser alteradas. En su lugar, se generan nuevas estructuras, eliminando problemas de concurrencia.
- Funciones como ciudadanos de primer orden: Las funciones pueden ser asignadas a variables, pasadas como argumentos y retornadas por otras funciones (funciones de orden superior).
- Evaluación perezosa (Lazy evaluation): Las expresiones no se evalúan hasta que su valor es estrictamente necesario, permitiendo el manejo de estructuras de datos infinitas y optimización de recursos.
- Transparencia referencial: Una función llamada con los mismos argumentos siempre devolverá el mismo resultado, facilitando la *eliminación programática de efectos colaterales* y permitiendo la demostración matemática de la corrección del código (Abelson et al., 1996).

- Paradigma Lógico / Declarativo

Se aleja totalmente de la arquitectura subyacente para centrarse en la lógica matemática de primer orden.

- Programación basada en relaciones: El desarrollador declara axiomas, hechos y reglas que describen el problema ("qué" es el problema), no los pasos para resolverlo.

- Unificación y Resolución de cláusulas de Horn: El motor de inferencia interno (como en Prolog) utiliza el algoritmo de unificación para encontrar valores que satisfagan variables libres en expresiones lógicas limitadas a cláusulas de Horn (reglas con a lo sumo un literal positivo).
- Abstracción total del flujo de control: El orden de ejecución es delegado al motor de inferencia del lenguaje subyacente, eximiendo al programador de gestionar bucles o sentencias de salto (Mitchell, 2003).

- Paradigma Concurrente / Actores (Emergente)

Diseñado específicamente para la era del procesamiento multinúcleo y sistemas distribuidos.

- Modelos de paso de mensajes: En lugar de compartir memoria, las entidades concurrentes (Actores) se comunican enviándose mensajes inmutables de forma asíncrona.
- Aislamiento estricto de estado: Cada actor mantiene su propio estado privado. Ningún otro hilo o actor puede modificar este estado directamente.
- Mitigación de condiciones de carrera a nivel de diseño lingüístico: Al eliminar la memoria compartida mutable (como se evidencia en lenguajes como Erlang, Elixir o en la gestión de *ownership* en Rust), el lenguaje garantiza por diseño la ausencia de *data races*, un problema crítico en sistemas concurrentes clásicos (Hewitt, 2010).

Análisis Paradigmático: El Fenómeno de la Convergencia Multiparadigma

En la ingeniería de software contemporánea, la pureza paradigmática ha dado paso a la convergencia multiparadigma pragmática. Los lenguajes modernos de amplio propósito (como Python, JavaScript, C#, Java, Swift y Rust) han evolucionado adoptando características cruzadas para responder a las exigencias extremas del mercado actual: procesamiento de Big Data, computación en la nube e interfaces de usuario altamente reactivas.

- La adopción de lo Funcional en contextos Imperativos/POO

El cambio más notable en la última década ha sido la inclusión de características del paradigma funcional en lenguajes fuertemente orientados a objetos.

- Problema de la industria: La necesidad de paralelizar cálculos en procesadores multinúcleo hace que la mutabilidad del estado (típica del paradigma imperativo/POO) sea peligrosa y propensa a **deadlocks** y condiciones de carrera.
- Solución multiparadigma: Java introdujo **Streams** y expresiones **Lambda**; C# adoptó **LINQ**; JavaScript fortaleció sus métodos de arreglos (``map``, ``filter``, ``reduce``). Esto permite a los desarrolladores encapsular la lógica en objetos (POO) pero transformar colecciones de datos utilizando tuberías inmutables y funciones puras (Funcional).
- Combinación de Actores y Declaratividad

Lenguajes modernos y frameworks aplican conceptos declarativos para el diseño de interfaces (ej. React, SwiftUI, Jetpack Compose) mientras utilizan bajo el capó la gestión imperativa para optimizar el rendimiento. Asimismo, la integración del paso de mensajes (Actores) o "canales" (ej. **GoRoutines** en Go) permite la construcción de microservicios concurrentes sin los bloqueos típicos de los modelos orientados a objetos puros.

Actividad II: Benchmarking Y Análisis Multiparadigma

Para evaluar de manera cuantitativa el impacto de los mecanismos de ejecución, los paradigmas de programación y la gestión de memoria, se diseñó un experimento cerrado de alta carga computacional. El algoritmo consiste en la resolución secuencial de una ecuación de segundo grado aplicada sobre arreglos dinámicos de tamaño $N = 200$, ejecutando un ciclo de procesamiento masivo de *100,000* iteraciones.

El entorno de pruebas utilizado para la recopilación de métricas fue el sistema operativo **Linux Mint**, garantizando la homogeneidad en la asignación de recursos del procesador y la memoria RAM.

Códigos Fuente Analizados

- **Implementación en Python (Paradigma Multiparadigma - Orientado a Objetos / Dinámico)**

Python prioriza la legibilidad y el dinamismo. En este código, la gestión de memoria es completamente automatizada y las variables no poseen tipos fijos en tiempo de compilación.

```
import math
import time

def calcular_cuadratica_vectores():
    n = 200
    # Inicialización homogénea de vectores (datos estables)
    a = [2.0] * n
    b = [5.0] * n
    c = [2.0] * n
    x = [0.0] * n

    iteraciones = 100000

    start_time = time.perf_counter()

    for _ in range(iteraciones):
        for i in range(n):
            discriminante = b[i]**2 - 4 * a[i] * c[i]
            x[i] = (-b[i] + math.sqrt(discriminante)) / (2 * a[i])

    end_time = time.perf_counter()
```



```

tiempo_ms = (end_time - start_time) * 1000
print(f"Tiempo de ejecución (Python): {tiempo_ms:.2f} ms")

if __name__ == "__main__":
    calcular_cuadratica_vectores()

```

El uso de listas nativas de Python implica que cada elemento es un objeto independiente en memoria, lo que destruye la localidad espacial en el caché de la CPU. Además, evaluar el ciclo *for* requiere que el intérprete verifique el tipo de dato en cada iteración, ralentizando el proceso.

- **Implementación en JavaScript / Node.js (Paradigma Multiparadigma - Basado en Prototipos)**

JavaScript emplea un entorno gestionado de ejecución sobre el motor V8, el cual aplica optimizaciones en tiempo de ejecución.

```

function calcularCuadraticaVectores() {
    const n = 200;
    const a = new Float64Array(n).fill(2.0);
    const b = new Float64Array(n).fill(5.0);
    const c = new Float64Array(n).fill(2.0);
    const x = new Float64Array(n);

    const iteraciones = 100000;

    const startTime = performance.now();

    for (let k = 0; k < iteraciones; k++) {
        for (let i = 0; i < n; i++) {
            let discriminante = (b[i] * b[i]) - (4 * a[i] * c[i]);
            x[i] = (-b[i] + Math.sqrt(discriminante)) / (2 * a[i]);
        }
    }

    const endTime = performance.now();
    const tiempoMs = endTime - startTime;
    console.log(`Tiempo de ejecución (JavaScript): ${tiempoMs.toFixed(2)} ms`);
}

calcularCuadraticaVectores();

```

Al utilizar *Float64Array*, obligamos al motor V8 a alojar los datos de forma contigua en la RAM. El bucle es detectado como un "hot path" (código caliente), por

lo que el compilador JIT (Just-In-Time) lo traduce rápidamente a código de máquina, aumentando drásticamente la velocidad respecto a Python.

- **Implementación en Rust (Paradigma Concurrente / Funcional / Imperativo)**

Rust destaca por su enfoque de compilación Ahead-Of-Time (AOT) y su estricto modelo de seguridad en memoria mediante reglas de propiedad (*ownership*), eliminando la necesidad de un recolector de basura.

```
use std::time::Instant;

fn main() {
    let n = 200;
    let a: Vec<f64> = vec![2.0; n];
    let b: Vec<f64> = vec![5.0; n];
    let c: Vec<f64> = vec![2.0; n];
    let mut x: Vec<f64> = vec![0.0; n];

    let iteraciones = 100000;

    let start_time = Instant::now();

    for _ in 0..iteraciones {
        for i in 0..n {
            let discriminante = (b[i] * b[i]) - (4.0 * a[i] * c[i]);
            x[i] = (-b[i] + discriminante.sqrt()) / (2.0 * a[i]);
        }
    }

    let duration = start_time.elapsed();
    println!("Tiempo de ejecución (Rust): {:.2} ms", duration.as_secs_f64() * 1000.0);

    // Esta línea obliga al compilador a no borrar el bucle
    println!("Control de optimización (Resultado en x[0]): {}", x[0]);
}
```

La compilación con la bandera de optimización máxima (*--release*) permite al backend LLVM vectorizar las operaciones matemáticas y mapear los arreglos directo en los registros del procesador. El compilador asegura que no existan accesos fuera de límites en tiempo de ejecución de forma ultra eficiente.

- **Implementación en Zig (Paradigma Imperativo / Estructurado de Sistemas)**

Zig ofrece un control absoluto del hardware y la memoria. Carece de variables ocultas o flujos de control complejos, exigiendo que la gestión de memoria y el tipado sean explícitos en el diseño.

```
const std = @import("std");
const time = std.time;
const math = std.math;

pub fn main() !void {
    const n = 200;

    // En Zig la gestión de memoria es explícita. Usamos el asignador nativo de Linux.
    const allocator = std.heap.page_allocator;

    const a = try allocator.alloc(f64, n);
    const b = try allocator.alloc(f64, n);
    const c = try allocator.alloc(f64, n);
    var x = try allocator.alloc(f64, n); // Este sí se queda como var porque modificamos
    sus elementos dentro del bucle

    // Aseguramos liberar la memoria al terminar el programa
    defer allocator.free(a);
    defer allocator.free(b);
    defer allocator.free(c);
    defer allocator.free(x);

    @memset(a, 2.0);
    @memset(b, 5.0);
    @memset(c, 2.0);

    const iteraciones = 100000;

    const start_time = time.nanoTimeTimestamp();

    var k: usize = 0;
    while (k < iteraciones) : (k += 1) {
        var i: usize = 0;
        while (i < n) : (i += 1) {
            const discriminante = (b[i] * b[i]) - (4.0 * a[i] * c[i]);
            x[i] = (-b[i] + math.sqrt(discriminante)) / (2.0 * a[i]);
        }
    }

    const end_time = time.nanoTimeTimestamp();
    const tiempo_ms = @as(f64, @floatFromInt(end_time - start_time)) /
    1_000_000.0;

    const stdout = std.io.getStdOut().writer();
    try stdout.print("Tiempo de ejecución (Zig): {d:.2} ms\n", .{tiempo_ms});
```

```
try stdout.print("Control de optimización (Resultado en x[0]): {d:.1}\n", .{x[0]});
}
```

Zig destaca por su rigidez; por ejemplo, si una variable declarada con `var` no muta su dirección, el compilador genera un error, forzando el uso preventivo de `const`. Compilado con `-Doptimize=ReleaseFast`, el rendimiento es puramente nativo y carece de cualquier capa intermedia (*overhead*).

Tabla de Benchmarking (Métricas Reales de Linux Mint)

A continuación, se consolidan las métricas empíricas obtenidas. El tiempo de ejecución se determinó mediante las APIs internas de alta precisión de cada lenguaje, mientras que el consumo de memoria física real se capturó mediante el comando del kernel `/usr/bin/time -v`, aislando el *Maximum Resident Set Size*.

Lenguaje de Programación	Paradigma Dominante	Mecanismo de Ejecución y Compilación	Tiempo de Ejecución Promedio (ms)	Consumo de Memoria Pico (MB)
Python	Multiparadigma (Dinámico)	Interpretado (Máquina Virtual / Bytecode)	12183.66 ms	9.62 MB
JavaScript	Multiparadigma (Prototipos)	Compilación JIT (Motor V8)	687.31 ms	51.45 MB
Rust	Concurrente / Funcional / Imperativo	Compilación AOT Nativa (Optimizado por LLVM)	135.82 ms	1.98 MB
Zig	Imperativo / Estructurado	Compilación AOT Nativa (Backend LLVM)	196.50 ms	0.25 MB

Análisis Técnico de Resultados

El análisis comparativo de las métricas recopiladas permite extraer deducciones críticas fundamentales para la ingeniería de software:

- **La Paradoja del Rendimiento JIT (JavaScript frente a Python):** Los resultados arrojan un contraste drástico entre ambos entornos gestionados. JavaScript demostró ser aproximadamente 28 veces más veloz que Python (687.31 ms frente a 12183.66). Esto se debe a la efectividad de la compilación *Just-In-Time* (JIT) del Motor V8, que traduce bloques calientes de código dinámico directamente a instrucciones nativas en la RAM durante la ejecución. No obstante, este mecanismo introduce una fuerte penalización de recursos: Node.js requirió 51.45 MB de memoria RAM, el consumo más elevado de toda la práctica, evidenciando el peso de la infraestructura y el almacenamiento en caché de código optimizado.
- **La Eficiencia Absoluta de la Compilación AOT (Zig y Rust):** Los lenguajes con compilación nativa *Ahead-Of-Time* (AOT) demostraron una superioridad incontestable tanto en velocidad como en austeridad de recursos. En esta evaluación, Rust se consolidó como el entorno más veloz de toda la práctica con un tiempo de ejecución sobresaliente de 135.82 ms y un consumo de memoria de apenas 1.98MB, validando que su modelo estricto de comprobación de punteros, reglas de propiedad (*ownership*) y gestión de ciclos de vida en tiempo de compilación provee un rendimiento idéntico al de los lenguajes de sistemas tradicionales sin añadir peso (*overhead*) al software en ejecución. Por su parte, Zig demostró un minimalismo extremo al coronarse como el sistema más eficiente en el uso de hardware, requiriendo un consumo imperceptible de 0.25 MB de memoria RAM física y un tiempo de procesamiento de 196.50ms. Esto se fundamenta firmemente en su filosofía de diseño: la ausencia total de variables ocultas, flujos de control complejos o un entorno de ejecución (*runtime*) pesado, interactuando de forma directa y quirúrgica con la arquitectura del procesador.

1. **El Costo Teórico de la Abstracción en Python:**

Python exhibió el tiempo más prolongado de la comparativa, superando los 10 segundos. Al depender de una interpretación lineal fundamentada en bytecode a través de su Máquina Virtual (CPython), cada acceso a los índices de las listas y cada operación matemática requiere una validación

dinámica del tipo de variable. Esto genera un constante cuello de botella en los ciclos de reloj del procesador. Sin embargo, su consumo de memoria RAM se mantuvo moderado 9.62 MB, limitándose estrictamente al tamaño de la máquina virtual base y la instanciación de los punteros de los objetos en la lista.

Actividad III : Diseño de un Lenguaje de Dominio Específico (DSL) para Sistemas lógico físicos Críticos

Se plantea el diseño e implementación conceptual de un Lenguaje de Dominio Específico (DSL) denominado Lenguaje L, orientado de manera exclusiva a la optimización, control y automatización de la planta de microredes inteligentes ECO-GRID.

A diferencia de los lenguajes de propósito general (como C++ o Java), un DSL restringe su expresividad matemática y sintáctica para ofrecer una abstracción directa y de alto nivel sobre los componentes de un ecosistema particular. En el caso de ECO-GRID, el lenguaje permite a los operadores e ingenieros interactuar directamente con actuadores, relés, sensores térmicos y líneas de transferencia sin necesidad de gestionar llamadas de bajo nivel a sistemas operativos o controladores de hardware (drivers).

Para el desarrollo de esta actividad, la arquitectura del lenguaje fue segmentada equitativamente en dos grandes bloques operativos:

- Bloque de Gestión de Planta y Control de Emergencias Críticas (Caso A).
- Bloque de Almacenamiento, Balance de Carga y Comercialización Eléctrica (Caso B).

Con el objetivo de mantener una cohesión e identidad lingüística unificada dentro del mismo compilador, se establecieron una fase de unificación morfológica preliminar: el uso estricto de identificadores bajo el formato snake_case para llamadas nativas, delimitadores lógicos explícitos en idioma español (inicio, fin, mientras), comentarios precedidos por doble barra (//) y variables de estado físico

parametrizadas exclusivamente mediante cadenas en mayúsculas sostenidas (CONECTADO y AISLADO).

Escenario A: Control de Planta y Gestión de Emergencias

El rol arquitectónico asumido en el Caso A contempla la abstracción conceptual de la infraestructura física base de ECO-GRID: arreglos de paneles solares fotovoltaicos, aerogeneradores térmicos y sensores de celda para el monitoreo de temperatura en tiempo real.

La finalidad del algoritmo diseñado es resolver una de las amenazas físicas más críticas en plantas de almacenamiento de litio: la fuga térmica (thermal runaway), un fenómeno donde el incremento descontrolado de la temperatura interna de una celda genera una reacción en cadena que destruye el los implementos de la planta.

Especificación de la Morfología Léxica (Tabla de Tokens)

La primera fase del compilador procesa el código fuente carácter por carácter para agruparlo en los siguientes componentes léxicos válidos y descartar la información innecesaria:

Token / Categoría	Patrón / Expresión Regular	Ejemplo en Código	Descripción Técnica
PR_CONTROL	mientras inicio fin	mientras, inicio, fin	Palabras reservadas que estructuran las sentencias y delimitan el bloque iterativo del lenguaje.
PR_ESTADO	CONECTADO AISLADO	CONECTADO, AISLADO	Literales de estado obligatorios (en mayúsculas) para controlar el comportamiento de los actuadores físicos y relés.
ID_FUNCION	[a-z_][a-z0-9_]*	leer_temperatura, conmutar_linea	Identificadores en formato snake_case que mapean llamadas nativas hacia las APIs del driver de hardware.
ID_DISPOSITIVO	[a-z_][a-z0-9_]*	bat_01, cooling_sys, panel_array	Variables e identificadores de componentes lógicos o sensores específicos dentro de la planta.
OP_RELACIONAL	>	>	Operador binario de comparación lógica (Mayor que).
LIT_ENTERO	[0-9]+	55	Constante numérica entera utilizada para definir límites y umbrales de seguridad.
LIT_CADENA	"[^"\\"]*"	"ALERTA CRÍTICA..."	Mensaje literal de texto delimitado por comillas dobles para reportes en la interfaz HMI.
DELIMITADOR	\(\) ;	(,) ;	Símbolos de agrupación para paso de argumentos y marcador de fin de instrucción lineal.
COMENTARIO	\\.*	//	Activar...Secuencias de texto ignoradas por el compilador durante la fase de análisis.

Reglas de Orden Gramatical (Sintaxis Formal)

Para proveer rigurosidad formal al compilador y evitar cualquier tipo de ambigüedad sintáctica en la estructura recursiva, se diseñaron las reglas de producción del lenguaje mediante la notación **EBNF (Extended Backus-Naur Form)**:

1. `<"programa" > ::= "\"init_grid;\\" <"lista_instrucciones" >`
2. `<"lista_instrucciones" > ::= <"instruccion" > [ <"lista_instrucciones" > ]`
3. `<"instruccion" > ::= <"sentencia_mientras" > | <"llamada_funcion" > "\";\\"`
4. `<"sentencia_mientras" > ::= "\"mientras\\" "\"(\\" <"condicion" > "\")\\"
\"inicio\\" <"lista_instrucciones" > "\"fin\\"`
5. `<"condicion" > ::= <"llamada_funcion" > "\"textgreater\\" "LIT_ENTERO"`
6. `<"llamada_funcion" > ::= "ID_FUNCION" "\"(\\" <"argumentos" > "\")\\"`
7. `<"argumentos" >::="ID_DISPOSITIVO"["\",\\" ("ID_DISPOSITIVO" |
PR_ESTADO" | "LIT_CADENA")]`

Se encarga de transformar el flujo continuo de caracteres del código fuente en unidades discretas con significado lógico denominadas tokens.

Se crearon 3 archivos, sean el “escenario.Ing” que contiene el código en Lenguaje L, “mi_analizador.py” que contiene los verificadores para que el lenguaje L pueda ser procesado y compilado con éxito, y el archivo [“tasks.js”](#) que tiene los comandos de lectura de archivos

Resultado de la corrida

```
14 // 3. Desviar el consumo del sector industrial hacia la red comercial de respaldo

PROBLEMAS SALIDA CONSOLA DE DEPURACIÓN TERMINAL PUERTOS QUERY RESULTS

PS C:\Users\PC1\OneDrive\Documentos\GitHub\LC-VisionLy\Tema2\Actividad 3> python mi_analizador.py escenario_a.lng
=====
INICIANDO COMPILADOR SIMULADO PARA ECO-GRID
Leyendo archivo de entrada: escenario_a.lng
=====

[1] FASE LEXICA: Escaneando alfabeto y construyendo tokens...
-> Línea 4: Procesando instruccion valida.
-> Línea 6: Procesando instruccion valida.
-> Línea 9: Procesando instruccion valida.
-> Línea 12: Procesando instruccion valida.
-> Línea 15: Procesando instruccion valida.
-> Línea 18: Procesando instruccion valida.
-> Línea 20: Procesando instruccion valida.
-> Línea 23: Procesando instruccion valida.

[2] FASE SINTACTICA: Validando gramatica abstracta de bloques...
-> Estructura mientras (... ) inicio ... fin' verificada exitosamente.

[3] RESULTADO DE EJECUCIÓN DEL HMI:
=====
>> [ALERTA HMI]: ALERTA CRITICA: Fuga termica en bat_01. Protocolo de alivio activo.
>> [SISTEMA]: Activando sistema de refrigeración auxiliar (Rele: CONECTADO).
>> [SISTEMA]: panel_array configurado en modo AISLADO.
>> [SISTEMA]: sector_industrial desviado a RED_COMERCIAL (CONECTADO).
=====
¡Análisis y ejecución finalizada con éxito!
PS C:\Users\PC1\OneDrive\Documentos\GitHub\LC-VisionLy\Tema2\Actividad 3> |
```

Escenario B:

Definición de Componentes Físicos Controlados (Sub-módulo de Almacenamiento y Comercialización)

- **Bancos de Baterías de Litio de Respaldo:** Unidades de almacenamiento de excedentes de carga del sistema ECO-GRID.
- **Sensores de Flujo Eléctrico y Caudal de Carga (Inversores):** Hardware de medición de entrada y salida energética expresada en kilovatios (kW).
- **Conmutadores Electrónicos de Red (Relés de Alta Potencia):** Actuadores mecánicos que permiten conectar o aislar sectores de consumo de la red comercial principal o de la microred local.

Especificación del Alfabeto y Reglas Léxicas

El submódulo lógico-comercial del Lenguaje L reconoce los siguientes tokens válidos para la toma de decisiones:

- **Palabras clave de lectura (Sensores):** estado_carga, leer_flujo.

- **Palabras clave de acción (Actuadores):** conmutar_linea, inyectar_red.
- **Operadores Relacionales y Lógicos:** > (mayor que), < (menor que), == (igual a), Y (conjunción lógica).
- **Tipos de datos aceptados:** Valores enteros (porcentajes del 0 al 100), valores decimales (flujo en kW) y literales booleanos de estado (CONECTADO, AISLADO).

Gramática Sintáctica Abstracta

Para el control de flujo basado en condiciones comerciales y de emergencia, el lenguaje implementa una estructura condicional secuencial. El orden gramatical estricto exigido por el compilador es el siguiente:

si condición_lógica entonces acciones_a_ejecutar sino_si
condicion_logica_alternativa entonces acciones_a_ejecutar fin_si.

Captura de pantalla del Lenguaje L en Archivo .txt

```
// LENGUAJE L - CONTROL DE ALMACENAMIENTO Y COMERCIALIZACIÓN
// Escenario Operativo B: Balance de Carga y Optimización Autónoma
// Desarrollado por: Genesis Moya C.I 31.370.339

// 1. LECTURA DE SENSORES Y ESTADO DEL SISTEMA
nivel_bateria = estado_carga(banco_litio_1)
generacion_actual = leer_flujo(inversor_paneles_solares)
demanda_actual = leer_flujo(inversor_consumo_planta)

// 2. EVALUACIÓN LÓGICA Y TOMA DE DECISIONES

// Caso A: Batería óptima y generación excede demanda (Inyección a red pública)
si (nivel_bateria > 90) Y (generacion_actual > demanda_actual) entonces
    // Conectar el relé principal hacia la red eléctrica pública
    conmutar_linea(rele_red_publica, CONECTADO)
    // Ordenar la inyección del excedente
    inyectar_red(CONECTADO)

// Caso B: Batería en límite crítico y sin generación solar (Emergencia nocturna)
sino_si (nivel_bateria < 20) Y (generacion_actual == 0) entonces
    // Aislar sectores no esenciales para preservar energía
    conmutar_linea(rele_linea_produccion, AISLADO)
    conmutar_linea(rele_oficinas_administrativas, AISLADO)

    // Garantizar suministro en áreas críticas
    conmutar_linea(rele_area_medica, CONECTADO)
    conmutar_linea(rele_servidores_criticos, CONECTADO)
fin_si
```

Captura de pantalla del Lenguaje L en Python

```

1 # SIMULADOR DE ANÁLISIS SEMÁNTICO - LENGUAJE L
2 # Escenario Operativo B: Balance de Carga y Optimización
  Autónoma
3 # Desarrollado por: Genesis Moya C.I 31.370.339
4
5 # --- DEFINICIÓN LÉXICA (Constantes) ---
6 CONECTADO = True
7 AISLADO = False
8
9 # --- ACTUADORES SIMULADOS ---
10 ▾ def conmutar_linea(sector, estado):
11     accion = "CONECTADO" if estado else "AISLADO"
12     print(f"[ACCION SINTACTICA] Relé '{sector}' ha sido
        {accion}")
13
14 ▾ def inyectar_red(estado):
15 ▾     if estado:

```

```

16         print("[ACCION SINTACTICA] Inyectando excedente a la
            red pública")
17
18 # --- 1. LECTURA DE SENSORES (Ingreso de datos en vivo) ---
19 print("\n=== LECTURA DE SENSORES ECO-GRID ===")
20 nivel_bateria = float(input("Ingrese el nivel de bateria de
    'banco_litio_1' (0-100%): "))
21 generacion_actual = float(input("Ingrese la generacion de
    'inversor_paneles_solares' (kW): "))
22 demanda_actual = float(input("Ingrese la demanda de
    'inversor_consumo_planta' (kW): "))
23
24 print("\n=== EVALUANDO ARBOL DE SINTAXIS (AST) ===")
25
26 # --- 2. EVALUACIÓN LÓGICA Y TOMA DE DECISIONES ---
27
28 # Caso A: Batería óptima y generación excede demanda
29 ▾ if (nivel_bateria > 90) and (generacion_actual > demanda_actual

```

```

    ):
30     print("-> FLUJO DE CONTROL: Caso A (Excedente de energia)")
31     conmutar_linea("rele_red_publica", CONECTADO)
32     inyectar_red(CONECTADO)
33
34     # Caso B: Batería en límite crítico y sin generación solar
    (Emergencia nocturna)
35 ▾ elif (nivel_bateria < 20) and (generacion_actual == 0):
36     print("-> FLUJO DE CONTROL: Caso B (Emergencia nocturna)")
37     conmutar_linea("rele_linea_produccion", AISLADO)
38     conmutar_linea("rele_oficinas_administrativas", AISLADO)
39
40     conmutar_linea("rele_area_medica", CONECTADO)
41     conmutar_linea("rele_servidores_criticos", CONECTADO)
42 |
43 ▾ else:

```

```

44     print("-> FLUJO DE CONTROL: Estado intermedio. No se
        requieren acciones criticas.")
45     print("=====\n")

```

Resultado de la corrida

=== LECTURA DE SENSORES ECO-GRID ===

Ingrese el nivel de batería de 'banco_litio_1' (0-100%): 15

Ingrese la generacion de 'inversor_paneles_solares' (kW): 0

Ingrese la demanda de 'inversor_consumo_planta' (kW): 150

=== EVALUANDO ARBOL DE SINTAXIS (AST) ===

-> FLUJO DE CONTROL: Caso B (Emergencia nocturna)

[ACCION SINTACTICA] Relé 'rele_linea_produccion' ha sido AISLADO

[ACCION SINTACTICA] Relé 'rele_oficinas_administrativas' ha sido
AISLADO

[ACCION SINTACTICA] Relé 'rele_area_medica' ha sido CONECTADO

[ACCION SINTACTICA] Relé 'rele_servidores_criticos' ha sido CONECTADO

=====

CONCLUSIÓN

El desarrollo de esta investigación permite consolidar una visión crítica sobre la infraestructura lingüística y computacional que sustenta al software moderno. Al analizar el fenómeno contemporáneo de la convergencia multiparadigma, se evidencia que la integración de características funcionales e imperativas no es una contradicción, sino una respuesta arquitectónica eficiente a las demandas de expresividad y concurrencia del mercado actual. Esta dualidad de diseño se ve reflejada directamente en el rendimiento técnico; las pruebas empíricas de *benchmarking* ratifican que los entornos compilados nativos como Zig y Rust, gracias a su control explícito de memoria y la ausencia de variables ocultas, logran una eficiencia en tiempos de ejecución y consumo de memoria pico que supera drásticamente a lenguajes dinámicos o interpretados como Python y JavaScript, los cuales heredan penalizaciones por la sobrecarga de sus motores en tiempo de ejecución o restricciones como el GIL.

Más allá de la eficiencia en la ejecución de algoritmos de procesamiento intensivo, el diseño abstracto del Lenguaje L para el ecosistema crítico ECO-GRID demuestra que la sintaxis y la morfología de un lenguaje son herramientas moldeables para dominios específicos. La especificación rigurosa de un alfabeto, tokens y reglas gramaticales conceptuales permite que la máquina interprete comandos de interfaces hombre-máquina sin ambigüedades, automatizando tareas de prevención térmica y balance de carga con total coherencia lógica. La transición del rol de programador aplicativo al de diseñador de lenguajes demuestra que cada decisión morfológica y sintáctica define la frontera entre un sistema vulnerable y una solución de software altamente optimizada, segura y adaptada a su entorno real.

REFERENCIAS

Abelson, H., Sussman, G. J., & Sussman, J. (1996). *Structure and interpretation of computer programs* (2.^a ed.). MIT Press.

Ghezzi, C., Jazayeri, M., & Mandrioli, D. (2002). *Fundamentals of software engineering* (2.^a ed.). Prentice Hall.

Hewitt, C. (2010). Actor model of computation: Scalable robust information systems. *Proceedings of the 7th International Conference on Informatics in Control, Automation and Robotics*, 1-12.

Llamas, L. (2026, 30 de enero). *Lenguajes compilados e interpretados: Diferencias*. Luis Llamas. <https://www.luisllamas.es/lenguajes-compilados-e-interpretados/>

MDN Web Docs. (2025, 10 de julio). *Float64Array - JavaScript*. Mozilla. https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Float64Array

Mitchell, J. C. (2003). *Concepts in programming languages*. Cambridge University Press.

Sebesta, R. W. (2019). *Concepts of programming languages* (12.^a ed.). Pearson.

Vista de aplicación de la metodología (JIT) Just in Time para aumentar la productividad en las empresas latinoamericanas. (s.f.). FitoVida. <https://revistas.unidx.edu.pe/index.php/FitoVida/article/view/11/11>